# AAD COURSE PROJECT REPORT

*on*

# TEXT/FILE SIMILARITY ANALYZER

## USING LONGEST COMMON SUBSEQUENCE

*by*

**Rohan Sandesh (U2304059)**

**Duvet B Srampickal (U2401077)**

**Abiya Susan Ray (U2404001)**

**Albin Rex (U2404002)**

*Submitted in partial fulfilment of the requirement*
*for the award of the degree of*

*Bachelor of Technology in Information Technology*




**Department of Information Technology**

**Rajagiri School of Engineering & Technology**

Rajagiri Valley P.O., Kakkanad, Kochi, Kerala - 682039

Academic Year 2026–2027

# ABSTRACT

The Text/File Similarity Analyzer is a Python desktop application that compares two typed text files using the Longest Common Subsequence (LCS) dynamic-programming algorithm. The application reads two UTF-8 files, converts their contents to lowercase, removes punctuation, and splits the text into words. It constructs a two-dimensional LCS table and backtracks through it to recover the longest ordered sequence of words shared by both files. A normalized LCS formula converts the result into a similarity percentage. The Tkinter interface displays both selected files, their word counts, LCS length, common word sequence, and final percentage. Testing produced 100 percent for identical normalized files and 84.85 percent for the supplied partially similar files. The project demonstrates a practical application of dynamic programming taught in Algorithm Analysis and Design.

# Contents

**Abstract** **i**

**1 INTRODUCTION** **1**

1.1 Background . . . . . 1

1.2 Problem Statement . . . . . 1

1.3 Objectives . . . . . 1

1.4 Scope . . . . . 1

**2 SYSTEM DESIGN** **2**

2.1 Tools and Technologies . . . . . 2

2.2 System Workflow . . . . . 2

2.3 Functional Modules . . . . . 2

**3 ALGORITHM AND IMPLEMENTATION** **3**

3.1 LCS Recurrence . . . . . 3

3.2 Text Normalization . . . . . 3

3.3 Similarity Formula . . . . . 3

**4 TESTING AND ANALYSIS** **4**

4.1 Test Results . . . . . 4

4.2 Complexity Analysis . . . . . 4

4.3 Limitations and Future Scope . . . . . 4

**5 CONCLUSION** **6**

**References** **7**

**Source Code** **8**

**A OUTPUT SCREENSHOTS** **13**

A.1 Identical Files . . . . . 13

A.2 Partially Similar Files . . . . . 14

# 1. INTRODUCTION

## 1.1 Background

A direct file-equality test only reports whether two files are exactly the same. It cannot express partial similarity or identify ordered content shared by two documents. This project solves that problem by applying Longest Common Subsequence at the word level. A subsequence preserves relative order but its elements do not need to be consecutive. Dynamic programming avoids repeatedly solving overlapping subproblems by storing intermediate results in a table.

## 1.2 Problem Statement

Develop a desktop application that accepts two text files, analyzes their ordered word-level similarity with LCS, and presents the common sequence and a percentage through a graphical interface.

## 1.3 Objectives

- Implement LCS manually using bottom-up dynamic programming.
- Accept two user-selected `.txt` files and preview their contents.
- Normalize capitalization, punctuation, and whitespace before comparison.
- Recover and display the longest common word sequence.
- Calculate and display a symmetric similarity percentage.
- Analyze the solution's time and space complexity.

## 1.4 Scope

The application compares UTF-8 typed text files at the word level. It is a similarity analyzer rather than a complete plagiarism detector because it does not understand synonyms, paraphrasing, or sentence meaning. Scanned pages and handwritten documents would require OCR or handwriting recognition and are outside the current AAD scope.

# 2. SYSTEM DESIGN

## 2.1 Tools and Technologies

**Table 2.1:** Tools used in the project

| **Component** | Purpose |
|---|---|
| **Python** | Implements normalization, file processing, LCS, and similarity calculation. |
| **Tkinter and ttk** | Create the desktop window, buttons, labels, previews, and results. |
| `filedialog` | Allows selection of the two input files. |
| `pathlib.Path` | Handles paths and reads UTF-8 file contents. |
| `re` | Removes punctuation with a regular expression. |

## 2.2 System Workflow

1. Start the application and select File 1 and File 2.
2. Read and preview both UTF-8 files.
3. Normalize both texts and create word lists.
4. Construct the LCS dynamic-programming table.
5. Backtrack to recover the common words.
6. Calculate the similarity percentage.
7. Display word counts, LCS length, sequence, and percentage.

## 2.3 Functional Modules

**Table 2.2:** Main functions

| **Function** | **Responsibility** |
|---|---|
| `normalize_text()` | Lowercases text, removes punctuation, and splits it into words. |
| `find_lcs()` | Builds the DP table and recovers the LCS through backtracking. |
| `calculate_similarity()` | Converts LCS length and word counts into a percentage. |
| `read_text_file()` | Reads a selected file using UTF-8 encoding. |
| `select_file1()/select_file2()` | Opens the file dialog and stores selected paths. |
| `compare_files()` | Coordinates normalization, LCS execution, and result display. |
| `clear_all()` | Clears selected files, previews, and previous results. |

# 3. ALGORITHM AND IMPLEMENTATION

## 3.1 LCS Recurrence

Let $X = (x_1, x_2, \ldots, x_m)$ and $Y = (y_1, y_2, \ldots, y_n)$ be the normalized word sequences. Define $dp[i][j]$ as the LCS length for the first $i$ words of $X$ and first $j$ words of $Y$.

$$dp[i][j] = \begin{cases} 0, & i = 0 \text{ or } j = 0, \\ dp[i-1][j-1] + 1, & x_i = y_j, \\ \max(dp[i-1][j], dp[i][j-1]), & x_i \neq y_j. \end{cases}$$

If the words match, they extend the best subsequence of both shorter prefixes. Otherwise, the algorithm keeps the better result obtained by excluding one current word. Backtracking from $dp[m][n]$ recovers the words in the optimal subsequence.

## 3.2 Text Normalization

Normalization converts both documents to lowercase, removes punctuation, and divides the remaining content into words. Consequently, capitalization and punctuation do not reduce the score, while word order remains significant.

## 3.3 Similarity Formula

If $L$ is the LCS word count and the two normalized files contain $m$ and $n$ words, the application uses:

$$\text{Similarity} = \frac{2L}{m+n} \times 100.$$

The formula gives 100 percent when the normalized sequences are identical and 0 percent when they contain no common word sequence.

# 4. TESTING AND ANALYSIS

## 4.1 Test Results

**Table 4.1:** Functional test cases

| No. | Input condition | Expected/observed result | Status |
|---|---|---|---|
| 1 | Identical normalized files | 100.00% similarity | Passed |
| 2 | Partially similar sample files | LCS 14; similarity 84.85% | Passed |
| 3 | One or both files missing | Warning requesting both files | Passed |
| 4 | Clear button selected | Inputs and outputs removed | Passed |
| 5 | Unreadable/invalid UTF-8 file | Error shown without program crash | Passed |

In the partial-similarity test, File 1 contained 17 words, File 2 contained 16 words, and the LCS contained 14 words:

$$\frac{2 \times 14}{17 + 16} \times 100 = 84.85\%.$$

Differences such as `subject/topic` and `by storing/using` were correctly excluded from the recovered common sequence.

## 4.2 Complexity Analysis

**Table 4.2:** Complexity of the implementation

| Operation | Time | Space |
|---|---|---|
| Normalization | $O(m + n)$ | $O(m + n)$ |
| DP-table construction | $O(mn)$ | $O(mn)$ |
| Backtracking | $O(m + n)$ | Output-dependent |
| Overall | $O(mn)$ | $O(mn)$ |

The nested loops visit every table cell once, while the $(m + 1)(n + 1)$ table dominates memory use.

## 4.3 Limitations and Future Scope

The analyzer measures ordered lexical overlap rather than meaning. Synonyms, paraphrasing, and substantial sentence reordering can lower the score. The $O(mn)$ table can also consume

considerable memory for very large files. Future enhancements may include common/unique-word highlighting, character and line comparison modes, exportable results, space-optimized LCS, support for additional document formats, and OCR as a separate preprocessing step for scanned documents.

# 5. CONCLUSION

The project successfully applies the Longest Common Subsequence dynamic-programming algorithm to a practical document-comparison problem. It reads and normalizes two text files, constructs and backtracks through the LCS table, and presents the common ordered content with a similarity percentage through an accessible Tkinter interface. The successful 100 percent and 84.85 percent tests confirm the implementation. The project clearly demonstrates overlapping subproblems, tabulation, reconstruction of an optimal solution, and complexity analysis from the AAD syllabus.

## REFERENCES


1. Module 3 Part 2 course notes, *Dynamic Programming and Longest Common Subsequence*.
2. Python Software Foundation, *tkinter — Python interface to Tcl/Tk*. Available at: https://docs.python.org/3/library/tkinter.html
3. Python Software Foundation, *pathlib — Object-oriented filesystem paths*. Available at: https://docs.python.org/3/library/pathlib.html
4. M. T. Goodrich, *Dynamic Programming: Longest Common Subsequences*, University of California, Irvine. Available at: https://ics.uci.edu/~goodrich/teach/cs260P/notes/LCS.pdf

# SOURCE CODE

The complete Python implementation is presented below. The listing is loaded directly from `main.py`, which must be kept in the same folder as this report file.

```python
import re
import tkinter as tk

from pathlib import Path
from tkinter import filedialog, messagebox, ttk


# Stores the paths of the selected files
file1_path = None
file2_path = None


def normalize_text(text):
    """
    Converts text to lowercase, removes punctuation
    and returns a list of words.
    """

    text = text.lower()
    text = re.sub(r"[^\w\s]", "", text)

    return text.split()


def find_lcs(words1, words2):
    """
    Finds the Longest Common Subsequence using
    dynamic programming.
    """

    m = len(words1)
    n = len(words2)

    # Create the DP table
    dp = [[0 for j in range(n + 1)] for i in range(m + 1)]

    # Fill the DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):

            if words1[i - 1] == words2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1

            else:
                dp[i][j] = max(
                    dp[i - 1][j],
                    dp[i][j - 1]
                )

    # Backtrack to recover the LCS words
    lcs_words = []

    i = m
    j = n

    while i > 0 and j > 0:

        if words1[i - 1] == words2[j - 1]:

            lcs_words.append(words1[i - 1])

            i -= 1
            j -= 1

        elif dp[i - 1][j] >= dp[i][j - 1]:

            i -= 1

        else:

            j -= 1

    # Reverse because words were collected backwards
    lcs_words.reverse()

    return lcs_words


def calculate_similarity(words1, words2, lcs_length):
    """
    Calculates the similarity percentage using
    the LCS length.
    """

    total_words = len(words1) + len(words2)

    if total_words == 0:
        return 100.0

    similarity = (2 * lcs_length / total_words) * 100

    return similarity


def read_text_file(path):
    """
    Reads a UTF-8 text file.
    """

    return Path(path).read_text(encoding="utf-8")


def select_file1():
    """
    Opens a dialog to select the first text file.
    """

    global file1_path

    selected_path = filedialog.askopenfilename(
        title="Select First Text File",
        filetypes=[
            ("Text files", "*.txt"),
            ("All files", "*.*")
        ]
    )

    # The user may cancel file selection
    if not selected_path:
        return

    file1_path = selected_path

    file1_label.config(
        text=Path(file1_path).name
    )

    display_file_content(
        file1_path,
        file1_text
    )


def select_file2():
    """
    Opens a dialog to select the second text file.
    """

    global file2_path

    selected_path = filedialog.askopenfilename(
        title="Select Second Text File",
        filetypes=[
```

```
            ("Text files", "*.txt"),
            ("All files", "*.*")
        ]
    )

    if not selected_path:
        return

    file2_path = selected_path

    file2_label.config(
        text=Path(file2_path).name
    )

    display_file_content(
        file2_path,
        file2_text
    )


def display_file_content(path, text_box):
    """
    Reads a file and displays its content
    inside a Tkinter text box.
    """

    try:
        content = read_text_file(path)

        text_box.config(state="normal")

        text_box.delete(
            "1.0",
            tk.END
        )

        text_box.insert(
            tk.END,
            content
        )

        text_box.config(state="disabled")

    except UnicodeDecodeError:

        messagebox.showerror(
            "File Error",
            "The selected file is not a valid UTF-8 text file."
        )

    except OSError as error:

        messagebox.showerror(
            "File Error",
            f"Unable to read the file.\n\n{error}"
        )


def compare_files():
    """
    Reads both selected files, applies LCS,
    and displays the similarity result.
    """

    if file1_path is None or file2_path is None:

        messagebox.showwarning(
            "Files Required",
            "Please select both text files."
        )

        return

    try:
        text1 = read_text_file(file1_path)
        text2 = read_text_file(file2_path)

    except UnicodeDecodeError:

        messagebox.showerror(
            "File Error",
            "One of the files is not a valid UTF-8 text file."
        )

        return

    except OSError as error:

        messagebox.showerror(
            "File Error",
            f"Unable to read the selected files.\n\n{error}"
        )

        return

    # Normalize both files
    words1 = normalize_text(text1)
    words2 = normalize_text(text2)

    # Find their LCS
    lcs_words = find_lcs(
        words1,
        words2
    )

    # Calculate similarity
    similarity = calculate_similarity(
        words1,
        words2,
        len(lcs_words)
    )

    # Display result
    similarity_label.config(
        text=f"{similarity:.2f}%"
    )

    word_count_label.config(
        text=(
            f"Words in File 1: {len(words1)}\n"
            f"Words in File 2: {len(words2)}\n"
            f"LCS Length: {len(lcs_words)}"
        )
    )

    lcs_result_text.config(state="normal")

    lcs_result_text.delete(
        "1.0",
        tk.END
    )

    if lcs_words:

        lcs_result_text.insert(
            tk.END,
            " ".join(lcs_words)
        )

    else:

        lcs_result_text.insert(
            tk.END,
            "No common word sequence found."
        )

    lcs_result_text.config(state="disabled")


def clear_all():
    """
    Clears both selected files and all results.
    """

    global file1_path
    global file2_path

    file1_path = None
    file2_path = None

```

```
    file1_label.config(
        text="No file selected"
    )

    file2_label.config(
        text="No file selected"
    )

    similarity_label.config(
        text="0.00%"
    )

    word_count_label.config(
        text=""
    )

    for text_box in [
        file1_text,
        file2_text,
        lcs_result_text
    ]:

        text_box.config(state="normal")

        text_box.delete(
            "1.0",
            tk.END
        )

        text_box.config(state="disabled")


# ------------------------------------------------
# Tkinter Window
# ------------------------------------------------

root = tk.Tk()

root.title(
    "Text/File Similarity Analyzer Using LCS"
)

root.geometry(
    "1000x700"
)

root.minsize(
    850,
    600
)

root.configure(
    background="#f4f6f8"
)


# ------------------------------------------------
# Styles
# ------------------------------------------------

style = ttk.Style()

style.configure(
    "Title.TLabel",
    font=("Arial", 22, "bold"),
    foreground="#17365d",
    background="#f4f6f8"
)

style.configure(
    "Heading.TLabel",
    font=("Arial", 12, "bold")
)

style.configure(
    "Similarity.TLabel",
    font=("Arial", 30, "bold"),
    foreground="#198754",
    background="#f4f6f8"
)


style.configure(
    "Action.TButton",
    font=("Arial", 11, "bold"),
    padding=8
)


# ------------------------------------------------
# Main Container
# ------------------------------------------------

main_frame = ttk.Frame(
    root,
    padding=20
)

main_frame.pack(
    fill="both",
    expand=True
)


# ------------------------------------------------
# Title
# ------------------------------------------------

title_label = ttk.Label(
    main_frame,
    text="Text/File Similarity Analyzer",
    style="Title.TLabel"
)

title_label.pack(
    pady=(0, 5)
)

subtitle_label = ttk.Label(
    main_frame,
    text="Using Longest Common Subsequence (LCS)"
)

subtitle_label.pack(
    pady=(0, 20)
)


# ------------------------------------------------
# File Selection Area
# ------------------------------------------------

files_frame = ttk.Frame(
    main_frame
)

files_frame.pack(
    fill="both",
    expand=True
)

files_frame.columnconfigure(
    0,
    weight=1
)

files_frame.columnconfigure(
    1,
    weight=1
)


# File 1 Area

file1_frame = ttk.LabelFrame(
    files_frame,
    text="File 1",
    padding=10
)

file1_frame.grid(
```

```
    row=0,
    column=0,
    padx=(0, 10),
    sticky="nsew"
)

select_file1_button = ttk.Button(
    file1_frame,
    text="Select File 1",
    command=select_file1,
    style="Action.TButton"
)

select_file1_button.pack(
    pady=5
)

file1_label = ttk.Label(
    file1_frame,
    text="No file selected"
)

file1_label.pack(
    pady=5
)

file1_text = tk.Text(
    file1_frame,
    height=13,
    wrap="word",
    font=("Arial", 10)
)

file1_text.pack(
    fill="both",
    expand=True,
    pady=5
)

file1_text.config(
    state="disabled"
)


# File 2 Area

file2_frame = ttk.LabelFrame(
    files_frame,
    text="File 2",
    padding=10
)

file2_frame.grid(
    row=0,
    column=1,
    padx=(10, 0),
    sticky="nsew"
)

select_file2_button = ttk.Button(
    file2_frame,
    text="Select File 2",
    command=select_file2,
    style="Action.TButton"
)

select_file2_button.pack(
    pady=5
)

file2_label = ttk.Label(
    file2_frame,
    text="No file selected"
)

file2_label.pack(
    pady=5
)

file2_text = tk.Text(
    file2_frame,
    height=13,
    wrap="word",
    font=("Arial", 10)
)

file2_text.pack(
    fill="both",
    expand=True,
    pady=5
)

file2_text.config(
    state="disabled"
)


# ------------------------------------------------
# Buttons
# ------------------------------------------------

button_frame = ttk.Frame(
    main_frame
)

button_frame.pack(
    pady=15
)

compare_button = ttk.Button(
    button_frame,
    text="Compare Files",
    command=compare_files,
    style="Action.TButton"
)

compare_button.grid(
    row=0,
    column=0,
    padx=10
)

clear_button = ttk.Button(
    button_frame,
    text="Clear",
    command=clear_all,
    style="Action.TButton"
)

clear_button.grid(
    row=0,
    column=1,
    padx=10
)


# ------------------------------------------------
# Result Area
# ------------------------------------------------

result_frame = ttk.LabelFrame(
    main_frame,
    text="Comparison Result",
    padding=15
)

result_frame.pack(
    fill="both",
    expand=True
)

similarity_heading = ttk.Label(
    result_frame,
    text="Similarity Percentage",
    style="Heading.TLabel"
)

similarity_heading.pack()

similarity_label = ttk.Label(
```

```
    result_frame,
    text="0.00%",
    style="Similarity.TLabel"
)

similarity_label.pack(
    pady=5
)

word_count_label = ttk.Label(
    result_frame,
    text="",
    justify="center"
)

word_count_label.pack(
    pady=5
)

lcs_heading = ttk.Label(
    result_frame,
    text="Longest Common Word Sequence",
    style="Heading.TLabel"
)

lcs_heading.pack(
    pady=(10, 5)
)

lcs_result_text = tk.Text(
    result_frame,
    height=5,
    wrap="word",
    font=("Arial", 10)
)

lcs_result_text.pack(
    fill="both",
    expand=True
)

lcs_result_text.config(
    state="disabled"
)


# Start the GUI
root.mainloop()
```

# A. OUTPUT SCREENSHOTS

## A.1 Identical Files

Figure A.1 shows the result for two identical normalized files. Both files contain 17 words, the LCS length is 17, and the reported similarity is 100.00 percent.

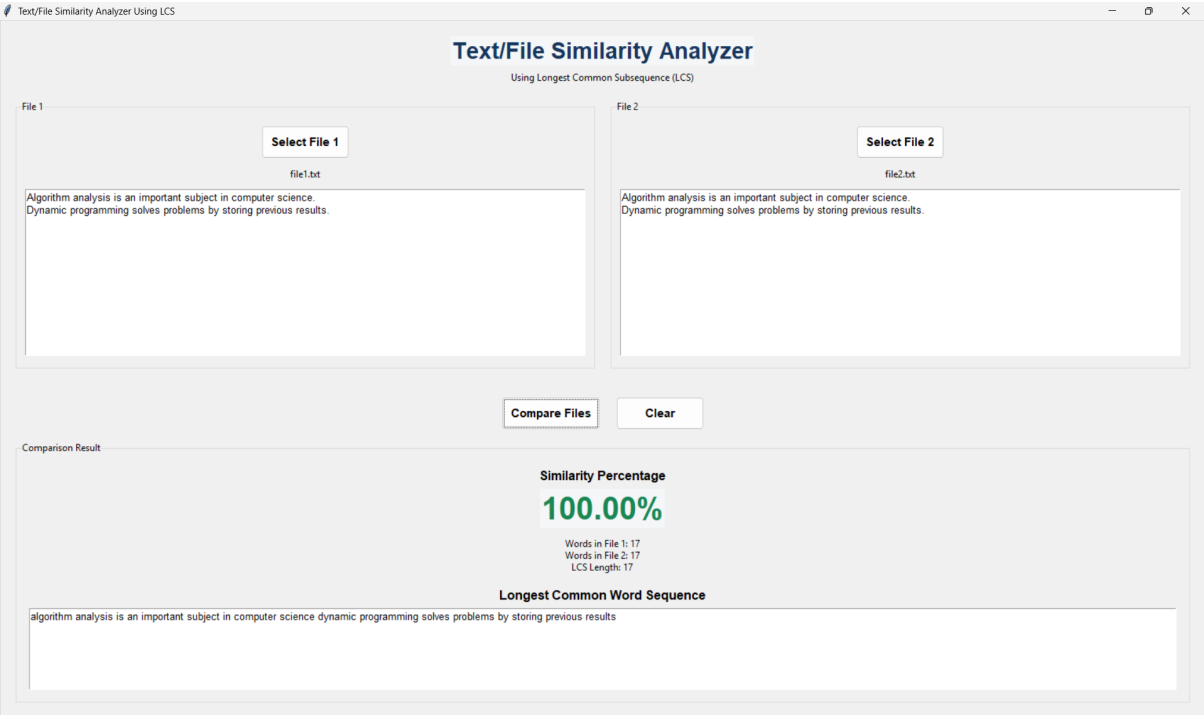



**Figure A.1:** Output for identical files showing 100.00% similarity

## A.2 Partially Similar Files

Figure A.2 shows the result for the partially similar files. File 1 contains 17 words, File 2 contains 16 words, the LCS length is 14, and the reported similarity is 84.85 percent.

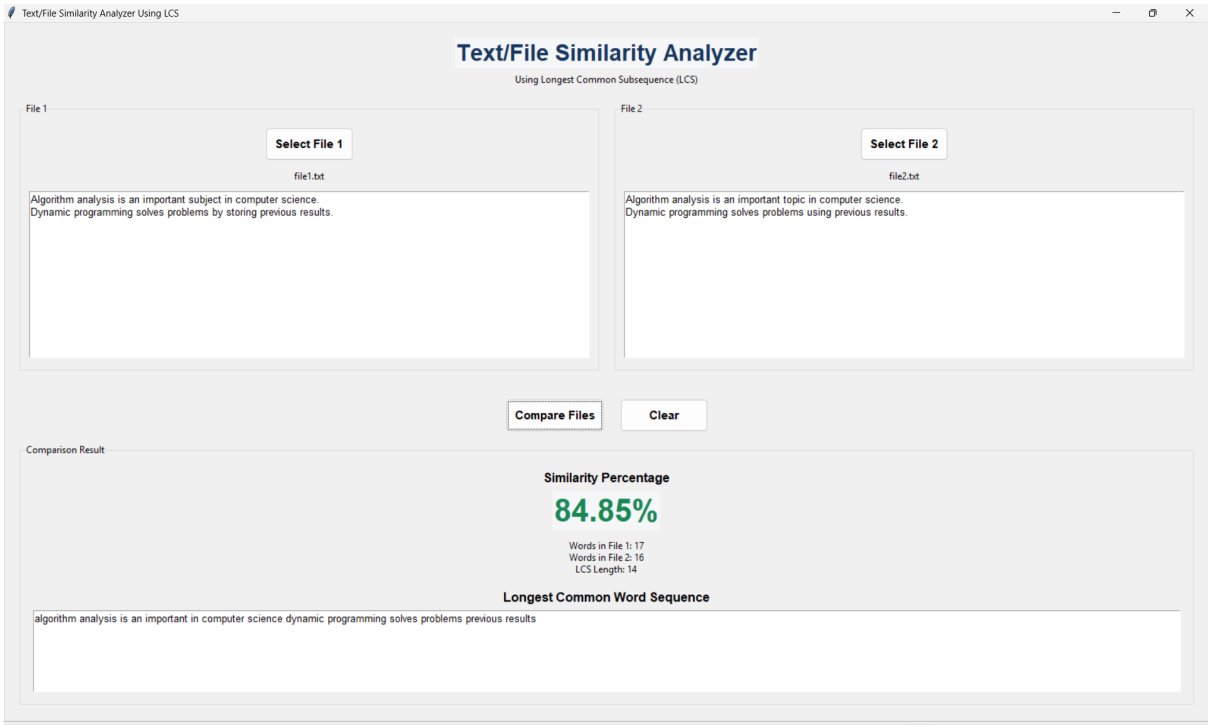



**Figure A.2:** Output for partially similar files showing 84.85% similarity